Del pedido en Jira a producción, con la spec como contrato

Spec-Driven Development (SDD): cada historia de Jira se convierte primero en una **especificación aprobada por el PO**; el código, las pruebas y las certificaciones se miden contra ella, y **controles automáticos bloquean** lo que no cumple.

Gerencia de Canales Digitales
Metodología SDD · noviembre 2026
Jira · GitHub Copilot · GitLab · Playwright

Negocio el PO, en Jira | **Célula** desarrollo asistido por IA | **Certificación** otras áreas, en QA | **Pase**

1 Pedido en Jira
El PO abre la épica (FUID, FDT o MAU) y la divide en historias con criterios de aceptación.

Refinada

2 Spec aprobada
La célula escribe la spec de la historia y sus dudas; el PO responde y la aprueba en Jira, antes de cualquier código.

Lista para desarrollo

3 Desarrollo con IA
Agentes de Copilot: pruebas primero, código después, revisión y seguridad. El desarrollador aprueba cada paso.

En desarrollo

4 Controles en GitLab
El pipeline bloquea el cambio si no cumple la spec, si se tocaron las pruebas o si falta la clave de Jira.

En desarrollo

5 Prueba de punta a punta
Pruebas automáticas de la funcionalidad completa en integración. Sin verde no se cierra la historia.

En integración

6 Certificación QA
QA certifica cada historia contra su spec y, después, la versión completa del pase.

En certificación QA

7 Rendimiento y seguridad
Sobre la versión completa: pruebas de carga con los umbrales del arquitecto y ethical hacking.

QA · Performance

Ethical hacking

✓ Producción
Despliegue por capas (MS → API → BFF → app) con la funcionalidad tras un interruptor. El PO la da por hecha.

Hecha

Sin los tres OK no hay pase, tampoco para un incidente urgente.

Un hallazgo de certificación es un **defecto enlazado a la historia**, se corrige con una prueba que lo reproduce.

El PO sigue el avance **en los estados de Jira**, sin entrar a ninguna otra herramienta.

Tres tipos de pedido, tres caminos

FUID — **Funcionalidad nueva o ampliada**
Cada pantalla o capacidad es una historia con su propia spec. Recorre el camino completo.

FDT — **Cambio puntual**
Actualizar un endpoint, migrar de versión. Regla de oro: si cambia el comportamiento, primero se actualiza la spec; si es solo técnico, regresión completa.

MAU — **Incidente en producción**
Diagnóstico, corrección y verificación por separado, con una prueba que reproduce el error antes de corregirlo.

Quién hace qué

PO	Pide, aclara y aprueba. Todo desde Jira.
Célula	4 devs full-stack: spec, código, pruebas y contratos. Dueña de sus repos.
Arquitecto	Define la carga y los umbrales de rendimiento.
QA	Certifica cada historia y la versión.
Performance	Ejecuta las pruebas de carga de BFF, API y MS.
Ethical hacking	Prueba la seguridad de la versión completa.

Organizados por funcionalidad

App propia | **BFF** propio | **API** o compartida | **MS** o compartido

- Unas **20 células** de 4 devs en Personas, Empresas y Adquirencia.
- Se trabaja por **funcionalidad**: Botón Pago Genérico, nóminas MTN, transferencias…
- Cada célula tiene **un repo** con las specs, los contratos y las pruebas de sus funcionalidades.

Qué gana la gerencia

- **Trazabilidad**: cada cambio se explica por una historia de Jira y su spec.
- **Calidad que no depende de personas**: los controles bloquean, no recomiendan.
- **Costo de IA por historia y por épica**, medido con la telemetría de Copilot.
- **Un solo proceso** en todas las células, sobre 18 arquetipos estándar.
- **Contratos versionados**: cambiar un servicio compartido no rompe otra funcionalidad.

Adopción en dos fases

Fase 1 · Piloto con los desarrolladores
Célula Hades (Botón Pago Genérico, Autorizar y Consultar Otros Comercios, botón de pago SII, nóminas): forma de trabajo con agentes en cada repo.

Fase 2 · Estándar de la gerencia
Repo por célula, controles que bloquean en GitLab, PO en Jira y tablero de costo de IA por historia y por equipo.